

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

PART IX

Stacks, Queues and Deques

LIFO and FIFO thinking — array and linked-list implementations, monotonic stacks, circular queues, and the STL containers that replace all of them in contest code

PART IX

Stacks, Queues and Deques

From “last in, first out” to monotonic structures that turn $O(n^2)$ scans into $O(n)$

Part VIII covered linked lists — structures where you can insert or remove almost anywhere. Stacks and queues are the opposite idea: structures that deliberately restrict where you can add or remove data, in exchange for speed, simplicity, and — as you'll see in this Part — the ability to solve a whole family of problems in linear time that would otherwise need nested loops. Restriction is the feature, not a limitation.

Contents of Part IX

- 1. What Are Stacks and Queues?
- 2. Stack — Array-Based Implementation
- 3. Stack — Linked-List Implementation and STL stack
- 4. Monotonic Stack
- 5. Queue and Circular Queue — Array-Based Implementation
- 6. STL queue and deque
- 7. Deque in Competitive Programming — Monotonic Deque
- Part IX Revision, Cheat Sheet, and Interview Questions

Chapter 1 — What Are Stacks and Queues?

1.1 Two disciplines for one job

A stack and a queue both do the same two things — add an element, remove an element — but they disagree completely about which element comes out. That single disagreement is the whole topic of this Part.

In one sentence: a stack removes the element that was added most recently (Last In, First Out — LIFO); a queue removes the element that was added least recently (First In, First Out — FIFO).

Real-world anchors

Stack: a physical stack of plates — you take the top plate off, and you put new plates on the top, never the bottom. Queue: a line at a ticket counter — whoever joined first is served first, and new people join at the back.

1.2 Where these show up without you naming them

Situation	Structure	Why
Function calls (call stack)	Stack	The most recently called function must finish, and return, before the one that called it can continue.
Undo in a text editor	Stack	The most recent action is the first one undone.
Browser back button	Stack	Most recently visited page is the first one you go back to.
Matching brackets / parentheses	Stack	The most recently opened bracket must be the next one closed.
Printer job queue	Queue	First document sent is the first one printed.
BFS traversal (Part XIII)	Queue	Nodes must be explored in the order they were discovered.

Situation	Structure	Why
CPU task scheduling (round-robin)	Queue	Fairness: process in arrival order.

1.3 The operations, precisely

Both structures expose a deliberately small interface. Restricting the interface is exactly what makes every operation $O(1)$ — there is never a need to shift or search, because there is only ever one legal place to add and one legal place to remove.

Operation	Stack	Queue
Add an element	push(x) — onto the top	push(x) / enqueue(x) — onto the back
Remove an element	pop() — from the top	pop() / dequeue() — from the front
Look without removing	top() — the top element	front() — the oldest element
Check if empty	empty()	empty()
All operations cost	$O(1)$	$O(1)$

Quick Revision

- Stack = LIFO (last in, first out); Queue = FIFO (first in, first out).
- Both expose only $O(1)$ operations at their designated ends — that restriction is what makes them fast.
- Recognize them by the problem's language: “most recent”/“nested”/“undo” suggests a stack; “arrival order”/“level by level” suggests a queue.

Chapter 2 — Stack: Array-Based Implementation

2.1 Building a stack on top of an array

A stack can be built on almost any underlying storage; an array (or, in modern C++, a vector) is the simplest and fastest, because push and pop only ever touch the end of the array — exactly where a vector already grows and shrinks cheaply.

```
class ArrayStack {
    vector<int> data;
public:
    void push(int x) { data.push_back(x); }
    void pop() { data.pop_back(); }           // caller must check
    empty() first
    int top() { return data.back(); }
    bool empty() { return data.empty(); }
    int size() { return data.size(); }
};
```

A minimal array-backed stack. In contest code you would almost always use `std::stack` or a plain vector directly instead of writing this class (Section 3.2) — this version exists to show what's happening underneath.

2.2 Line by line

`push_back` appends to the end of the vector — that end plays the role of the “top” of the stack. `pop_back` removes the last element in $O(1)$ (no shifting is needed, because nothing after it needs to move). `back()` reads the last element without removing it. Because all three operations act only on the end of the vector, none of them ever has to move any other element — that is the entire reason a stack is $O(1)$ per operation.

2.3 Dry run: balanced parentheses

The single most common stack problem: given a string of brackets, decide if every opening bracket has a matching, correctly-nested closing bracket.

```
bool isBalanced(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else {
            if (st.empty()) return false;           // closing with
            nothing open
            char top = st.top();
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) return false;
            st.pop();
        }
    }
    return st.empty();    // every opener must have been closed
}
```

Char read	Action	Stack after
(	push	(
[	push	([
]	top is '[', matches — pop	(
)	top is '(', matches — pop	(empty)

Input "`([])`" ends with an empty stack, so it is balanced. The key idea worth internalizing: a stack is the natural structure for “the thing that must close is always the most recently opened thing” — that phrase describes nesting in general, not just brackets.

⚠ Common mistake

Forgetting to check `st.empty()` before calling `st.top()` or `st.pop()` on an empty stack is undefined behaviour in C++ — it will not throw a clean exception like some other languages; it can silently corrupt memory or crash unpredictably. Always guard `pop()/top()` with an `empty()` check when the stack could plausibly be empty.

Practice — Chapter 2

Problem 2.1 — Reverse a string using a stack [Level 0]

Given a string, reverse it using an explicit stack (not built-in reverse).

Hint 1: Push every character, then pop them all — what order do they come out in?

Approach: Push each character onto a stack, then repeatedly pop and append to the result — LIFO order reverses the sequence.

Complexity: $O(n)$ time, $O(n)$ extra space.

Problem 2.2 — Valid parentheses with three bracket types

[Level 1]

Given a string containing '(', ')', '[', ']', '{', '}', determine if it is balanced.

Hint 1: This is exactly the dry-run example above — implement it yourself before checking.

Key observation: A closing bracket is only valid if it matches the type of the most recently opened, still-unclosed bracket — which is always the top of the stack.

Complexity: $O(n)$ time, $O(n)$ space.

Chapter 3 — Stack: Linked-List Implementation and STL stack

3.1 Why a linked list also works

A stack can equally be built on a singly linked list (Part VIII), by always inserting and removing at the head — never the tail. Inserting/removing at the head of a singly linked list is $O(1)$, which is exactly the requirement.

```
struct Node { int val; Node* next; };

class LinkedStack {
    Node* head = nullptr;
public:
    void push(int x) {
        Node* n = new Node{x, head};
        head = n;
    }
    void pop() {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
    int top() { return head->val; }
    bool empty() { return head == nullptr; }
};
```

A linked-list-backed stack. Correct, but rarely used directly in contests — array-backed storage has better cache locality and avoids per-node allocation overhead, so it is faster in practice despite identical Big-O complexity.

Array vs. linked-list stack — which to actually use

In competitive programming: always the array/vector version, or better, the STL `std::stack` (Section 3.2). The linked-list version matters conceptually (and sometimes in interviews, where it demonstrates you understand both underlying structures), but costs real speed in a contest with tight time limits due to memory allocation and pointer-chasing overhead.

3.2 std::stack

In every contest, you use the STL version. `std::stack` is what's called a container adapter — it doesn't store data itself; by default it wraps a deque (Section 6) and simply restricts which operations you're allowed to call on it.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    stack<int> st;
    st.push(10);
    st.push(20);
    st.push(30);
    cout << st.top() << "\n";    // 30
    st.pop();
    cout << st.top() << "\n";    // 20
    cout << st.size() << "\n";   // 2
    cout << st.empty() << "\n"; // 0 (false)
}
```

Operation	Meaning	Complexity
push(x)	add x to the top	O(1) amortized
pop()	remove the top element (returns nothing)	O(1)
top()	read the top element without removing it	O(1)
empty()	true if the stack has no elements	O(1)
size()	number of elements currently stored	O(1)

⚠ pop() returns nothing

Unlike some languages, `st.pop()` in C++ does not return the removed value — it only removes it. If you need the value, read it with `top()` first, then call `pop()` separately.

Practice — Chapter 3

Problem 3.1 — Evaluate a postfix expression [Level 2]

Given a postfix (Reverse Polish) expression like "2 3 4 * +", evaluate it using a stack.

Hint 1: Push numbers. When you see an operator, pop two numbers, apply the operator, push the result back.

Key observation: Postfix notation is designed exactly so that a single left-to-right pass with a stack evaluates it, with no need for parentheses or precedence rules.

Approach: For each token: if it's a number, push it. If it's an operator, pop b then a (in that order — order matters for - and /), push (a operator b). The final stack has exactly one element: the answer.

Complexity: $O(n)$ time, $O(n)$ space.

Chapter 4 — Monotonic Stack

4.1 The problem that motivates it

Consider: for every element of an array, find the next greater element to its right (or -1 if none exists). The brute-force approach checks, for every index, every index after it — $O(n^2)$. For n up to 10^5 , that is far too slow. A monotonic stack solves this exact shape of problem in $O(n)$.

4.2 What “monotonic” means here

A monotonic stack is an ordinary stack with one extra rule enforced on every push: before pushing a new element, pop off anything already on the stack that would break a chosen ordering (increasing or decreasing). The stack's contents are therefore always sorted — monotonic — at every point in time.

```
vector<int> nextGreaterElement(vector<int>& a) {
    int n = a.size();
    vector<int> ans(n, -1);
    stack<int> st;           // stores INDICES, kept decreasing
    in value
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) {
            ans[st.top()] = a[i]; // a[i] is the next greater element
            st.pop();             // whatever index is on top
        }
        st.push(i);
    }
    return ans; // indices still on the stack at the end have no next greater element; ans[] was initialized to -1
}
```

Next greater element to the right, for every index, in $O(n)$ total.

4.3 Dry run

Array: [2, 1, 2, 4, 3]. The stack stores indices; shown values are $a[\text{index}]$ for readability.

i	a[i]	Action	Stack (values) after	ans updated
0	2	stack empty, push	[2]	—
1	1	$1 < 2$, no pop, push	[2, 1]	—
2	2	$2 > 1$, pop (idx 1); $2 == 2$, stop; push	[2, 2]	ans[1] = 2
3	4	$4 > 2$, pop (idx 2), ans[2]=4; $4 > 2$, pop (idx 0), ans[0]=4; push	[4]	ans[2]=4, ans[0]=4
4	3	$3 < 4$, no pop, push	[4, 3]	—

Final answer: [4, 2, 4, -1, -1]. Indices 3 and 4 remain on the stack at the end with no greater element after them, so they keep their initial -1.

4.4 Why this is $O(n)$, not $O(n^2)$

The nested while loop looks worrying at first glance — but every index is pushed onto the stack exactly once and popped off at most once, across the entire run of the algorithm. Total push operations: n . Total pop operations: at most n . So total work across all iterations is $O(n)$, not $O(n)$ per iteration. This style of argument is called amortized analysis — an individual iteration's while loop might do more than $O(1)$ work, but the total across all iterations is still bounded by $O(n)$.

✓ How to recognize a monotonic-stack problem

Clue words: “next greater/smaller element,” “previous greater/smaller element,” “largest rectangle in a histogram,” “span”/“days until warmer temperature.” The common shape: for every element, you need the nearest element to the left or right satisfying some comparison — and a brute-force pairwise scan would be $O(n^2)$.

4.5 Variants at a glance

Goal	Stack direction	Scan direction	Pop condition
Next greater element	decreasing	left to right	$a[\text{top}] < a[i]$
Next smaller element	increasing	left to right	$a[\text{top}] > a[i]$
Previous greater element	decreasing	right to left	$a[\text{top}] < a[i]$
Previous smaller element	increasing	right to left	$a[\text{top}] > a[i]$

Practice — Chapter 4

Problem 4.1 — Daily temperatures [Level 3]

Given daily temperatures, for each day find how many days you'd have to wait until a warmer temperature (0 if never).

Hint 1: This is next-greater-element, but the answer stored is a distance ($i - \text{top index}$), not a value.

Key observation: Same monotonic-decreasing-stack-of-indices pattern as next greater element — only what you store in `ans[]` changes.

Approach: Maintain a decreasing stack of indices by temperature. When $a[i]$ beats the top, $\text{ans}[\text{top}] = i - \text{top}$, then pop.

Complexity: $O(n)$ time, $O(n)$ space.

Problem 4.2 (harder) — Largest rectangle in a histogram [Level 5]

Given bar heights of a histogram, find the area of the largest rectangle that fits entirely under the histogram outline.

Hint 1: For each bar, imagine it is the shortest bar in the rectangle. How wide can the rectangle extend left and right before hitting a shorter bar?

Hint 2: 'Nearest shorter bar to the left' and 'nearest shorter bar to the right' are both monotonic-stack problems.

Key observation: A monotonic increasing stack of indices lets you compute, for the bar being popped, exactly how far its rectangle could extend in $O(1)$ per bar.

Approach: Push indices while heights increase. When a shorter bar arrives, pop and compute area using (current index - new stack top - 1) as width, popped height as height.

Complexity: $O(n)$ time, $O(n)$ space — each index pushed and popped once.

Chapter 5 — Queue and Circular Queue: Array-Based Implementation

5.1 The naive array queue, and why it's wasteful

A first attempt at a queue on an array: keep a front index and a back index. Enqueue writes at back and increments it; dequeue simply increments front (leaving the old element in place, just no longer “in” the queue).

```
int arr[100], frontIdx = 0, backIdx = 0;    // backIdx = next free slot

void enqueue(int x) { arr[backIdx++] = x; }
int dequeue() { return arr[frontIdx++]; }    // caller must check non-empty first
```

⚠ The wasted-space problem

Every dequeue moves frontIdx forward but never reuses the slots before it. After many enqueue/dequeue cycles, backIdx can reach the end of the array even though most of the array is actually empty (just “used up” space to the left of frontIdx). A naive fixed array queue eventually reports “full” when it logically isn't.

5.2 The fix: a circular queue

A circular queue treats the array as a ring: when an index would run off the end, it wraps back to 0 using the modulo operator. This reclaims the space freed by dequeues without ever shifting elements.

```

class CircularQueue {
    vector<int> a;
    int frontIdx = 0, cnt = 0, cap;
public:
    CircularQueue(int capacity) : a(capacity), cap(capacity) {}

    bool full() { return cnt == cap; }
    bool empty() { return cnt == 0; }

    void enqueue(int x) {
        int backIdx = (frontIdx + cnt) % cap;    // next free slot,
        wrapped
        a[backIdx] = x;
        cnt++;
    }
    int dequeue() {
        int x = a[frontIdx];
        frontIdx = (frontIdx + 1) % cap;        // wrap around
        cnt--;
        return x;
    }
    int front() { return a[frontIdx]; }
};

```

5.3 Dry run (capacity 4)

Call	front	cnt	Array state (slots 0..3)	Note
enqueue(10)	0	1	[10, _, _, _]	backIdx was $(0+0)\%4=0$
enqueue(20)	0	2	[10, 20, _, _]	backIdx= $(0+1)\%4=1$
dequeue() → 10	1	1	[10, 20, _, _]	slot 0 now logically free
enqueue(30)	1	2	[10, 20, 30, _]	backIdx= $(1+1)\%4=2$
enqueue(40)	1	3	[10, 20, 30, 40]	backIdx= $(1+2)\%4=3$

Call	front	cnt	Array state (slots 0..3)	Note
enqueue(50)	1	4	[50, 20, 30, 40]	$\text{backIdx} = (1 + 3) \% 4 = 0$ — wraps, reuses slot 0

Slot 0's old value (10) is safely overwritten by 50 only after 10 was already dequeued — the wraparound reuses freed space instead of leaving it stranded, which is exactly what the naive version in 5.1 failed to do.

✓ **In a contest, you rarely write this by hand**

Understanding the circular-queue mechanism matters for interviews and for building other structures (some sliding-window and buffer problems) on top of it — but for a plain queue in a contest, `std::queue` (Chapter 6) already handles all of this internally.

Practice — Chapter 5

Problem 5.1 — Simulate a circular queue [Level 2]

Implement `enqueue`, `dequeue`, `front`, `isFull`, and `isEmpty` for a fixed-capacity circular queue, and trace it on a sequence of operations.

Hint 1: Track count explicitly (as in the code above) rather than trying to distinguish full from empty using only `frontIdx` and `backIdx` — both cases can otherwise look identical.

Approach: Use the `cnt` field to disambiguate a full ring from an empty one (both would otherwise have `frontIdx == backIdx`).

Complexity: $O(1)$ per operation.

Chapter 6 — STL queue and deque

6.1 std::queue

Like `std::stack`, `std::queue` is a container adapter — by default built on top of `deque` — that exposes only the FIFO-appropriate operations.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    queue<int> q;
    q.push(10);
    q.push(20);
    q.push(30);
    cout << q.front() << "\n";    // 10 (oldest)
    cout << q.back()  << "\n";    // 30 (newest)
    q.pop();
    cout << q.front() << "\n";    // 20
    cout << q.size()  << "\n";    // 2
}
```

Operation	Meaning	Complexity
<code>push(x)</code>	add x to the back	O(1) amortized
<code>pop()</code>	remove the front element (returns nothing)	O(1)
<code>front()</code>	read the oldest element	O(1)
<code>back()</code>	read the newest element	O(1)
<code>empty() / size()</code>	as expected	O(1)

6.2 std::deque — a queue open at both ends

`deque` (double-ended queue) supports O(1) insertion and removal at both ends — unlike `vector`, where inserting at the front is O(n) because every existing element must shift. Internally it's typically implemented as a sequence of fixed-size blocks rather than one contiguous array,

which is what makes front-insertion cheap.

```
deque<int> dq;
dq.push_back(10);
dq.push_back(20);
dq.push_front(5);
// dq is now [5, 10, 20]
cout << dq.front() << " " << dq.back() << "\n";    // 5 20
dq.pop_front();    // removes 5
dq.pop_back();     // removes 20
// dq is now [10]
cout << dq[0] << "\n";    // 10 – deque also supports random access
                             like vector
```

Container	Fast insert/erase at	Random access <code>dq[i]</code>	Typical use
vector	back only	yes, $O(1)$	default choice for a dynamic array
deque	front and back	yes, $O(1)$	need both ends fast (BFS, sliding window)
stack	top only (adapter)	no	pure LIFO logic
queue	front/back only (adapter)	no	pure FIFO logic

Why BFS (Part XIII) uses queue, not stack

Breadth-first search must visit nodes in the exact order they were discovered, level by level — that's a FIFO requirement, so BFS always uses a queue. Depth-first search, by contrast, naturally matches a stack's LIFO order (and is very often written with recursion instead, which uses the call stack implicitly — see Part X).

Practice — Chapter 6

Problem 6.1 — Implement a stack using two queues [Level 3]

Using only queue operations (push/pop/front), implement a stack's push and pop.

Hint 1: One classic approach: make push expensive — after adding the new element, rotate the rest of the queue behind it.

Approach: On `push(x)`: enqueue `x`, then dequeue-and-re-enqueue every other element in the queue exactly once, so `x` ends up at the front. Now `front()` of the queue always matches `top()` of the stack.

Complexity: $O(n)$ per push, $O(1)$ per pop — a deliberate trade-off to keep pop simple.

Chapter 7 — Deque in Competitive Programming: Monotonic Deque

7.1 The problem: sliding window maximum

Given an array and a window size k , find the maximum element in every contiguous window of size k as the window slides across the array.

Brute force recomputes the max of each window from scratch: $O(n \times k)$.

For n and k both up to 10^5 , that's far too slow. A monotonic deque solves it in $O(n)$.

7.2 The idea

Maintain a deque of indices, kept in decreasing order of their values, and restricted to indices that are still inside the current window. Two things need to happen at every step: remove indices that have fallen out of the window from the front, and remove indices whose value is smaller than the incoming element from the back (they can never be the maximum of any future window while a bigger, more recent element is still in play). This is exactly why it needs a deque and not a plain stack or queue — it must pop from both ends.

```

vector<int> maxSlidingWindow(vector<int>& a, int k) {
    deque<int> dq;                // stores indices, values kept
    decreasing
    vector<int> ans;
    for (int i = 0; i < (int)a.size(); i++) {
        if (!dq.empty() && dq.front() <= i - k)
            dq.pop_front();        // out of window

        while (!dq.empty() && a[dq.back()] <= a[i])
            dq.pop_back();        // smaller/equal, can't
win        now or later

        dq.push_back(i);

        if (i >= k - 1)
            ans.push_back(a[dq.front()]); // front is always the
            current max
    }
    return ans;
}

```

7.3 Dry run

Array: [1, 3, -1, -3, 5, 3, 6, 7], $k = 3$. Deque shown as index:value pairs.

i	a[i]	Deque after processing	Window max (once $i \geq k-1$)
0	1	[0:1]	—
1	3	pop 0:1 ($3 \geq 1$), [1:3]	—
2	-1	[1:3, 2:-1]	3
3	-3	[1:3, 2:-1, 3:-3]	3
4	5	front 1:3 out of window ($1 \leq 1$), pop; pop 2:-1, 3:-3 (≤ 5); [4:5]	5
5	3	[4:5, 5:3]	5

i	a[i]	Deque after processing	Window max (once $i \geq k-1$)
6	6	pop 5:3, pop 4:5 (both ≤ 6); [6:6]	6
7	7	pop 6:6 (≤ 7); [7:7]	7

Final answer: [3, 3, 5, 5, 6, 7]. Note the front of the deque is always the index of the current window's maximum — reading it is $O(1)$, no scanning required.

✓ How to recognize a monotonic-deque problem

Clue words: “sliding window,” combined with “maximum/minimum in every window.” The giveaway that separates it from a plain monotonic stack (Chapter 4): the window has both a fixed size and slides forward, so old elements must eventually be evicted from the front — that eviction is the reason a deque (not a stack) is required.

Quick Revision

- A monotonic deque keeps candidate indices in sorted-by-value order and needs eviction from BOTH ends: expired indices from the front, dominated values from the back.
- The front of the deque always holds the answer for the current window — read it in $O(1)$.
- Same amortized- $O(n)$ argument as the monotonic stack: each index is pushed once and popped at most once, across the whole run.

Practice — Chapter 7

Problem 7.1 — Sliding window minimum [Level 3]

Adapt the sliding window maximum algorithm to find the minimum of every window instead.

Hint 1: Only the comparison direction changes.

Approach: Keep the deque increasing in value instead of decreasing: pop from the back while $a[dq.back()] \geq a[i]$.

Complexity: $O(n)$ time, $O(k)$ space.

Problem 7.2 (harder) — Shortest subarray with a sum at least K [Level 5]

Given an array (which may contain negative numbers) and a target K , find the length of the shortest contiguous subarray whose sum is $\geq K$.

Hint 1: Work with the prefix-sum array P , where $P[i]$ is the sum of the first i elements. The problem becomes: find $i < j$ minimizing $j - i$ such that $P[j] - P[i] \geq K$.

Hint 2: Maintain a deque of indices into P , kept increasing in P -value, and prune from the front once a valid pair is found.

Key observation: Any index i with $P[i] \geq P[i']$ for some earlier $i' < i$ can never be a better left endpoint than i' — it's both farther away AND has a worse (larger) prefix value, so i' dominates it and i can be discarded.

Approach: Push j onto a deque of candidate left endpoints (kept increasing in P value). While $P[j] - P[\text{front}] \geq K$, that's a candidate answer — pop the front and keep shrinking. Before pushing j , pop from the back while $P[\text{back}] \geq P[j]$ (dominated).

Complexity: $O(n)$ time, $O(n)$ space — each index pushed and popped from the deque at most once.

Part IX — Master Revision

Must Remember

Quick Revision

- Stack = LIFO; all operations act on the top. Queue = FIFO; add at the back, remove from the front.
- In contest code, use `std::stack` and `std::queue` (or plain vector/deque) — never hand-write array/linked-list versions unless the problem specifically asks for it.
- A circular queue reuses freed space by wrapping indices with %, avoiding the naive array queue's “false full” problem.
- A monotonic stack solves “next/previous greater/smaller element” problems in $O(n)$ via amortized analysis: each element pushed once, popped at most once.
- A monotonic deque extends the same idea to sliding-window maximum/minimum problems, evicting from the front (expired) and the back (dominated) as needed.
- `std::deque` gives $O(1)$ insert/erase at both ends, unlike vector (front insert is $O(n)$ there).

Common Mistakes Recap

Mistake	Fix
Calling <code>top()/pop()</code> on an empty stack or queue	Always check <code>empty()</code> first when it's not provably non-empty
Assuming <code>st.pop()</code> returns the removed value	It returns void in C++ — read with <code>top()/front()</code> before popping
Using <code>vector::insert(begin(), x)</code> for front-insertion in a loop	Use deque instead — vector front-insertion is $O(n)$ per call
Writing a monotonic stack/deque without deciding increasing vs. decreasing up front	Fix the invariant first (what comparison triggers a pop?), then write the loop
Forgetting to evict expired indices from a monotonic deque's front	Check and pop-front before processing each new index, not after

Mistake	Fix
Naive fixed-array queue reporting 'full' with unused space	Use a circular queue (wrap with %) or just use <code>std::queue</code>

Complexity Table

Structure / operation	Complexity
stack / queue: push, pop, top/front, empty, size	$O(1)$ each
deque: push_front, push_back, pop_front, pop_back, operator[]	$O(1)$ each
Monotonic stack — building next/previous greater/smaller array	$O(n)$ total (amortized)
Monotonic deque — sliding window maximum/minimum	$O(n)$ total (amortized)
Brute-force window maximum (naive)	$O(n \times k)$

Interview-Style Questions

- Why are all core stack and queue operations $O(1)$, while an arbitrary-position insertion into a vector is $O(n)$?
- Why does a circular queue need to track a count (or equivalent) rather than just front and back indices?
- Explain, in your own words, why a monotonic stack achieves $O(n)$ total work despite a nested loop in its implementation.
- Why does the sliding window maximum problem need a deque specifically, rather than a stack or a plain queue?
- Why is `std::deque`, not `std::vector`, the default underlying container for `std::stack` and `std::queue`?

Cheat Sheet — Part IX

```

// Stack
stack<int> st;
st.push(x); st.pop(); st.top(); st.empty(); st.size();

// Queue
queue<int> q;
q.push(x); q.pop(); q.front(); q.back(); q.empty(); q.size();

// Deque
deque<int> dq;
dq.push_front(x); dq.push_back(x);
dq.pop_front(); dq.pop_back();
dq.front(); dq.back(); dq[i];

// Monotonic stack skeleton (next greater element)
stack<int> st; // indices, values decreasing
for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.top()] < a[i]) {
        ans[st.top()] = a[i];
        st.pop();
    }
    st.push(i);
}

// Monotonic deque skeleton (sliding window maximum)
deque<int> dq; // indices, values decreasing
for (int i = 0; i < n; i++) {
    if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
    while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
    dq.push_back(i);
    if (i >= k - 1) ans.push_back(a[dq.front()]);
}

```

Coming up in Part X

Part X — Recursion and Backtracking goes back to the call stack from Chapter 1 of this Part and studies it directly: how recursive calls actually behave on that stack, how to trace recursion depth and stack frames by hand, and how backtracking (permutations, subsets, N-Queens, pruning) builds on recursion to systematically explore a search space without missing or repeating possibilities.